BULLETTIME: Time Dilation for High-Fidelity Tracing

ISCA 2026 Submission **#720** – Confidential Draft – Do NOT Distribute!!

***Abstract*—Much of computer systems and architecture research depends on accurate, high-fidelity program tracing for simulation, profiling, and debugging. Unfortunately, with significant improvements in compute and memory instruction execution speeds, tracing frameworks suffer from frequent I/Os associated with persisting traced events to disk. We find that such delays can be bursty and asymmetric across application threads, resulting in an inadvertent reordering of application and system operations relative to untraced execution. In our analysis, such reordering often results in significant changes to the studied application behavior, polluting any insights that may be drawn from simulation and profiling studies on the corresponding captured traces.**

In this work, we formalize the notion of studied application behavior to establish correctness requirements for traced application execution with tracing-induced delays. We propose a novel *time dilation* approach that strategically slows down the execution for application and system threads in a way that meets correctness requirements. We realize the time dilation approach in BULLETTIME, a tracing framework built atop Pin, the de facto binary instrumentation tool. We evaluate BULLETTIME for memory contiguity and synchronization studies over real-world applications and workloads. Our results show that while existing tracing approaches can cause application behavior to deviate by as much as $20\times$ compared to untraced execution, BULLETTIME's deviations are $< 10\%$ even in extreme cases of asymmetric tracing delays.

I. INTRODUCTION

Application tracing is widely used in computer systems research. It provides insights into workload characteristics [12], [21], [34], [35], [45], drives software simulator models of proposed hardware [8], [26], [36], [40], [50], [51], [53], and even guides real-system prototypes of new software systems [29], [44], [52]. Tracing is powerful because it can selectively record systems interactions and events. In principle, high-fidelity tracing should track not just application execution, but also accurately capture how applications interact with diverse systems software components (e.g., kernel scheduling, networking, virtual memory, compilers, etc.), strengthening the conclusions drawn from trace-based studies.

Unfortunately, as processing and memory speeds grow faster, the overheads injected by the tracing framework become a key hurdle to extensive use of tracing. A contributor to such overheads is the I/O required to persist the traced information itself, exacerbated by the fact that today, I/O speeds lag behind memory and CPU instruction execution (and corresponding tracing) speeds. In practice, this results in frequent stalls in the traced application threads to flush out the traced information to disk. While prior approaches have explored reducing overheads associated with tracing [23], [24], [48], we posit that such overheads cannot be eliminated—some I/O is necessary to collect traced information. We also find that tracing-induced delays have a more concerning outcome than application slowdown—they can change the behavior of the traced application altogether (§II).

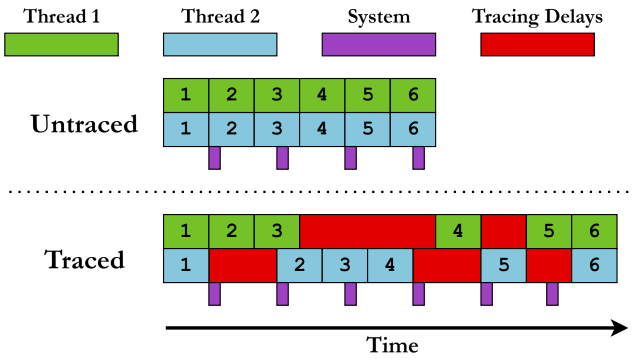

Fig. 1: The effect of tracing delays on interleaving of application threads and the surrounding system. Each numbered block represents a unit of application work (*operation*). Purple blocks denote periodic invocations of system daemons. Tracing injects bursty delays that are asymmetric across application threads and system daemons. Delay bursts misalign application threads from each other and the system, resulting in the application deviating from its untraced behavior.

In particular, tracing-induced I/O tends to be asymmetric across the traced application threads. For instance, if a tracing study only focuses on instrumenting memory instructions in an application, its threads will experience time-varying amounts of I/O based delay, depending on the frequency of memory access across threads and over time. This effect is illustrated in Figure 1, where operations performed across the different threads get reordered in time, changing the observed impact of those operations throughout the application's execution. Moreover, these delays also cause system tasks (e.g., kernel daemons) that affect application behavior to be reordered relative to the application's execution, since these daemons will not suffer any delays due to tracing. An immediately noticeable impact in Figure 1 is that the traced application will experience disproportionately more system activity than an untraced execution, since its execution is longer with tracing.

While our analysis holds for any study that relies on high-fidelity program tracing, in this work, we examine this problem specifically for memory contiguity and synchronization studies as case studies — fields where accurate tracing is paramount, since they operate at the boundary of hardware and software.

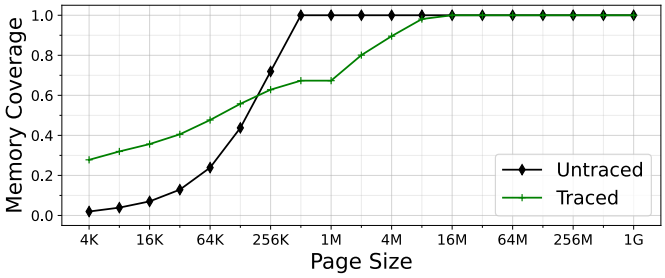

Fig. 2: CDF of memory coverage by page size for a key-value store in traced and untraced executions. The traced execution overestimates the portion of the application's memory footprint covered by regions with little contiguity.

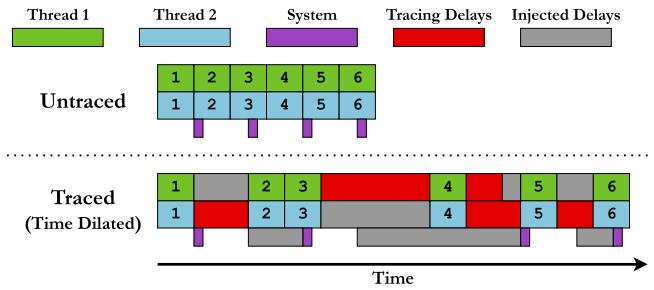

Fig. 3: Restoring application behavior in traced execution using *time dilation*. Application behavior is restored if traced application threads and system threads are slowed down to match the pace of the slowest thread.

The tracing studies in these fields are useful not only in developing efficient and robust software mechanisms (e.g., testing and developing race-free software [29], [43], [44], [54] using synchronization traces) but also in designing efficient hardware and operating systems (e.g., changes in production TLB hardware and memory management sub-systems [15], [39], [41], [47] using contiguity traces). Moreover, modern operating systems often rely on system daemons to detect and generate memory contiguity [2], [53], [55], making such studies particularly vulnerable to behavioral divergence due to tracing delays.

Empirically, we find that tracing effects can significantly affect the application behavior under study. Figure 2 shows the cumulative distribution of memory coverage by page size for a key-value store workload when run in traced and untraced configurations. To quantify contiguity, we calculate the minimum number of pages required to cover each physically contiguous region of memory, where pages can be sized at any power-of-2 above the default 4KB. This process maximizes the amount of memory covered by a hypothetical TLB design [39], [40], where the use of power-of-2 pages is under active research [37]. Our results indicate that traced execution presents a misleading picture of memory contiguity, suggesting that about half of the contiguity is present in small groups of pages (<64KB). Such misrepresentations can mislead the development of new architectures, such as a TLB that is overoptimized to coalesce smaller contiguous regions. Similarly, errant behavior is seen in other workloads and metrics of interest (see §V), hinting at the scope of the issue uncovered in this work.

Ensuring application behavior is preserved with tracing first requires identifying the correct behavior for an execution. A straightforward definition of correct execution would require all application and system threads to execute all operations in the same order as they would have in the untraced execution. Unfortunately, striving for such a correctness definition is intractable to achieve for a real application deployed on a system with hundreds to thousands of executing components. On the other extreme, manually selecting traced components to constrain the modeling complexity of correctness may miss components such as obscure system daemons that rarely affect application behavior.

Our approach to defining correctness starts with a precise description of the application behavior we want to study, and then identifying the components that must execute correctly (§III-A). At the core are two ideas: the application's *key state*—the state we want to observe during execution—and its *key operations*—the operations, across both application and system threads, that modify this state.

Correctness then comes down to two simple requirements. First, tracing must not introduce any new key operations of its own (i.e., it should not change application behavior). Second, the order of key operations across all relevant threads must remain the same with and without tracing. This definition keeps the focus on components that truly affect application behavior, while ignoring the many other threads/activities that do not impact the key state under study.

Equipped with a precisely-scoped definition of correctness centered on *key state*, we propose a novel *time dilation* approach to achieving it for traced application executions (§III-B). The idea underpinning this approach is that preserving the order of key operations across various threads experiencing asymmetric tracing delays in a traced execution requires slowing down (*time dilating*) all threads to match the speed of the slowest thread (i.e., the one with the most tracing delays). Figure 3 depicts this idea in the most ideal sense — it injects just the right amount of delay (shown in gray) across both application and system threads to make sure all of them execute at the same pace, thereby preserving the order of key operations and, consequently, correct behavior.

We realize the time-dilation approach in our Pin-based [31] tracing framework, BULLETTIME[1], which dynamically modulates the speed of application and system threads to ensure correct application behavior. BULLETTIME focuses explicitly on behavior changes due to asymmetric tracing delays from I/O for persisting traced information. Our practical realization of time dilation intercepts these I/O triggers to detect the

[1]Popularized by the movie "The Matrix", BULLETTIME is a cinematic technique that slows down time while the camera appears to move at normal speed around the scene. This creates the illusion that the viewer is moving freely around a moment that's unfolding in extreme slow motion.

extent of slowdowns in each application thread. Using the delay estimates, it artificially injects just enough delays during the intercepted I/Os to ensure the rate of execution for all application threads is consistent with that of the slowest thread. By identifying the kernel threads that perform key operations, BULLETTIME also slows down their execution by prolonging the duration that they `sleep()` for, so that their execution speed also matches that of the slowest thread over time. We select these mechanisms due to their prevalence across all Linux systems, allowing BULLETTIME to be widely deployable across many machines and architectures.

We evaluate BULLETTIME's ability to preserve application behavior for memory contiguity and synchronization studies atop real-world applications and workloads, while recording all memory-referencing instructions of an application. On a set of popular cloud workloads, BULLETTIME improves contiguity metrics by **2.3**× over existing tracing approaches, closely matching untraced execution. On a synchronization benchmark designed to maximize the difference in tracing delays between threads, while existing tracing approaches can deviate from untraced application behavior by as much as 20×, BULLETTIME ensures the deviation remains within 10% of untraced execution.

II. BACKGROUND AND MOTIVATION

A. Delays in Tracing Frameworks

It is well known that tracing incurs non-trivial runtime overhead to an application's execution, much of which is unavoidable when capturing low-level hardware events. In most real-world scenarios, the primary source of the overhead in fine-grained tracing is the I/O required to store the traced events. Figure 4 shows, for memory tracing, how the trace data generation rate scales as a function of memory references per instruction. Even when tracing just a single thread, the trace data rate exceeds the storage bandwidth of our test system at only one memory reference per *twenty* instructions. For a more realistic baseline, the benchmarks used in our evaluation commonly reference memory one *out of every three* instructions. Moreover, since trace data rates scale with the number of threads in the traced application, incoming data can quickly overwhelm even sophisticated storage systems, even with multiple GB/s of write bandwidth. Because bandwidth oversubscription is so high, asynchronous I/O is of little help, as the system will quickly revert to synchronous behavior once all buffers are filled. The observable impact of such high trace data rates is non-uniform tracing-induced delays across application threads.

Although prior approaches have explored reducing overheads associated with tracing [23], [24], [48], we are unaware of works that have studied the impact of these tracing-induced delays on changes to traced application behavior. Such changes threaten to undermine the usefulness of the trace for any simulation, profiling, or debugging applications [5], [7], [12], [21], [22], [34]–[36], [46]. The focus of this work, therefore, is to understand potential changes in application behavior due to tracing and to develop mechanisms to preserve behavior with

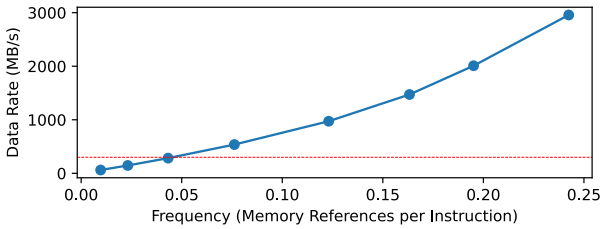

Fig. 4: Trace data generation rate for a given rate of memory-referencing instructions for a single thread. The data rate quickly outpaces the storage bandwidth (in red) of our test system (§V), being an order of magnitude higher at only one reference every four instructions. Our benchmarks commonly make one reference every three instructions for each thread.

tracing. Indeed, the compute, memory, and I/O scaling trends discussed above suggest that this problem is likely to only get more prominent in the foreseeable future, especially for tracing fine-grained hardware activity. We next explain, using real-world tracing studies, the exact mechanisms by which tracing can modify application behavior.

B. Understanding Behavior Changes due to Tracing

We now illustrate through two use cases how the tracing infrastructure can itself interfere with the application's executions in subtle ways to change the behavior being studied. We will use these use cases as running examples and evaluation candidates for the rest of the paper.

Memory contiguity studies. Memory contiguity studies focus on an application's virtual-to-physical memory mappings at various parts of its execution, typically to understand the length and number of contiguous physical memory allocations. These studies are critical to the effective use of hugepages [2], [41], [53] as well as efficient TLB designs [39], [40]. A common approach for such studies is the use of tracing tools to track memory access patterns (i.e., `LOAD` and `STORE` instructions) and application memory allocations, while periodically observing the virtual-to-physical memory mappings (e.g., via the page table). Unfortunately, the tracing framework itself performs certain operations that cause the virtual-to-physical memory allocations to change relative to an untraced execution.

We identify two main categories of "culprit" tracing operations. The first — and the more obvious — category comprises operations that directly affect the virtual-to-physical mappings. These include memory allocations and deallocations that the tracing framework must perform at runtime to manage its own memory, which inadvertently changes the physical memory available to the application itself.

The second category affects the mappings more subtly, and is related to the delays introduced by the tracing infrastructure's operations — we illustrate this through the example portrayed in Figure 1. For this example, assume that the two threads each perform memory allocation as their fourth operation. In this case the desynchronization between each of

TABLE I: Example program where a synchronization study that traces only memory operations can make `memops()` slower than `computeops()`, making it likely that `interesting()` is never executed with tracing, unlike untraced execution.

```
int x = 0;
mutex lock;
```

// Thread #1	// Thread #2
memops();	computeops();
acquire(lock);	acquire(lock);
if (x == 0)	x = 1;
interesting();	release(lock);
release(lock);	

the threads themselves and between threads and the system daemons means the first thread will see a different layout of application data structures than in the untraced execution, changing its allocation behavior[2]. Moreover, if the system daemon performs meaningful work, such as collapsing a hugepage or compacting memory, both threads will see a different physical memory layout during block 4, which will change key behavior.

Synchronization studies. Another class of studies where tracing is critical involves synchronization effects. These studies capture traces of synchronization-heavy multi-threaded applications, so that they can be simulated at a later time for cost-efficiency [44], understanding performance bottlenecks [29], detecting race conditions [54], etc. As with contiguity studies, tracing can inject its operations into such multi-threaded applications, resulting in a change in synchronization behavior. Again, these tracing operations fall into the same two categories: those that inject synchronization operations of their own (which are rare), and those that inject delays to change how synchronization occurs.

The latter, unfortunately, is not only common but can be notoriously hard to detect and can even change behavior to the extent of changing application outputs. Table I illustrates this with a simple example, where two threads are synchronized by a single mutex (`lock`). The entry into the critical section is gated by a memory-operation-intensive function for the first thread, and a compute-intensive one for the second thread. Without any tracing, if both functions are likely to take the same time to execute, then the first thread has a roughly 50% chance of executing the `interesting()` function, depending on which thread enters the critical section first. Now consider execution under a tracing framework that only traces memory operations, disproportionately slowing down the execution of `memops()` compared to `computeops()`. This biases the execution in a way that ensures that the second thread would enter the critical section first, eliminating the possibility of the first thread executing `interesting()`. While this is only a simple example, identifying such code paths can be extremely difficult in large codebases with many conditional executions protected by synchronization — common in large-scale multi-threaded cloud frameworks like key-value stores [4], [10], [19].

[2]The same occurs for the fourth operation of the second thread, due to the misalignment of its second and third operations.

III. Preserving Application Behavior with Time Dilation

We begin by describing our approach to modeling the application behavior both with and without tracing and formalizing the requirements for tracing to preserve application behavior relative to native execution (§III-A). We then describe our *time dilation* approach, which carefully dilates the execution of application and system components to preserve application behavior under tracing (§III-B).

A. Modeling the Application Behavior

A key challenge in modeling application behavior when tracing stems from appropriately *scoping* the application and system components that are responsible for said behavior. Returning to our memory contiguity studies — at a high level, contiguity is affected broadly by memory allocations and deallocations. If we scope the components that affect contiguity too narrowly, e.g., by only considering allocations/deallocations from the application, we risk ignoring important "full system" effects, e.g., periodic compactions of regular pages into huge pages by Linux's `khugepaged` daemon [2]. On the other extreme, if we define our scope too broadly, we would end up modeling components not relevant to contiguity (e.g., modern Linux has tens of daemons that have no bearing on memory contiguity), resulting in an exponential blow-up in modeling complexity.

To this end, our approach relies on a precise definition of application behavior that the tracing study is interested in, which is then used to identify the exact set of application and system (i.e., OS or kernel) components that may affect said behavior. More concretely, we introduce two concepts central to the application behavior under study: *key state* and *key operations*.

The key state, s, refers to the state critical to defining application behavior, and its contents depend on the focus of the tracing study. For instance, in a memory contiguity study, s must include the layout of free physical memory and the application's virtual-to-physical mappings. If these traces are used for cache and TLB simulations, then s should also include the layout of the application's allocated data structures. A key operation *op*, on the other hand, is any event that *modifies* key state, i.e, an application of this operation leads to a modified key state, denoted as:

$$s' \leftarrow op(s)$$

Key operations comprise all instructions that modify the contents of key state. Continuing our memory contiguity example, key operations would occur during all memory allocations and deallocations, as well as any memory migrations performed by kernel daemons (e.g., those performed by the `khugepaged`).

A salient feature of this model is that we ignore all other forms of state and operations in either the application or the system, since perturbations to them are inconsequential to

the tracing study. This allows us to constrain the scope of our problem to a well-defined, complete set of interactions between the application, system, and tracing framework.

Modeling the untraced application execution. The untraced ("native") application execution is modeled as an ordered list of key operations, Ops, comprising key operations from both the application and system threads. Since these threads execute concurrently, the order of operations in Ops is defined by the serialized order of their execution in the untraced run. More formally, $\mathsf{Ops} = \{op_1, op_2, \ldots, op_n\}$, where i denotes the order of op_i's execution. We use $\mathsf{Thread}(op_i)$ to denote the thread that executes op_i. If s_0 is the initial key state, then the *application behavior* under native execution is captured in the ordered sequence of key state versions, $\mathcal{S} = \{s_0, s_1, s_2, \ldots, s_n\}$, where the next key state version is obtained by applying the next operation on the current state, i.e., $s_{i+1} \leftarrow op_{i+1}(s_i)$.

Modeling traced application execution. The same application, when traced, is modeled as a similar list of operations, $\mathsf{Ops}^t = \{op_1^t, op_2^t, \ldots, op_m^t\}$. As with the untraced execution, $\mathsf{Thread}(op_i^t)$ denotes the thread that executes op_i^t. There are, however, two key differences between Ops^t and Ops. First, Ops^t contains *additional* operations (denoted as the ordered list $\mathcal{T}$) which correspond to operations performed by the tracing framework itself, i.e., $m \geq n$. Note that if the operations in $\mathcal{T}$ modify key state, then it changes application behavior relative to native execution. We also note that the remainder of the operations in Ops^t are still the same as the operations in Ops. Formally, if we denote $\hat{\mathsf{Ops}}$ as the list Ops^t with all operations performed by the tracing framework removed (i.e., $\hat{\mathsf{Ops}} = \mathsf{Ops}^t \setminus \mathcal{T}$), then $|\hat{\mathsf{Ops}}| = |\mathsf{Ops}|$. However, the second difference between the two lists is that the order of operations in Ops may not be preserved in $\hat{\mathsf{Ops}}$. As noted in §II, this re-ordering of operations — typically caused by delays injected by tracing operations ($\mathcal{T}$) themselves — is the other reason that tracing changes application behavior relative to native execution.

Correctness conditions. With the execution models defined above, we can now formally establish correctness conditions to ensure that application behavior is preserved during traced execution. These conditions effectively prevent the two sources of behavioral changes discussed above:

- **C1** The tracing framework should not modify the key state s. In other words, no operation in $\mathcal{T}$ is a key operation.
- **C2** The order of key operations in the traced execution should be the same as the order of operations in the native execution. Formally, $\hat{\mathsf{Ops}} = \mathsf{Ops}$.

Meeting the correctness condition **C1** requires modifying the tracing framework to prevent it from making any key state modifications. For many tracing studies, it is rare for the tracing framework to alter the key state, although not impossible — e.g., memory allocations/deallocations within the tracing framework affect the traced application's contiguity (as noted in §II). Fortunately, addressing this is somewhat straightforward — effectively removing any such operations from the tracing framework (e.g., by preallocating any memory buffers required by the tracing framework ahead of time for the tracing study). We save implementation details of how this is realized in our framework to §IV.

B. Time Dilation Approach

Our approach to address the sources of application behavior changes, termed *time dilation*, effectively ensures that the traced execution preserves the correctness condition **C2**.

Key idea. The time dilation approach strategically injects delays in traced execution across various threads, in a manner that ensures all key operations in the traced execution ($\hat{\mathsf{Ops}}$) occur in the same order as those for the untraced execution (Ops), ensuring condition **C2**. Our driving observation is that the changes in ordering of key operations across threads occur due to the imbalance in the delays caused by operations injected by the tracing framework (i.e., operations in $\mathcal{T}$) across various application and system threads. Therefore, to restore application behavior, the delays need to be made equitable across all threads. By injecting delays into the "faster" executing threads — threads with fewer delays due to tracing framework operations — we effectively *dilate* the notion of time in such threads until other threads catch up to them.

The "ideal" time dilation. While time dilation can correct the application behavior under study, any delay injected into the traced threads unfortunately increases the tracing overhead by forcing longer executions. It is, however, possible to minimize the amount of delay that must be injected to ensure correct behavior by meeting condition **C2**. This ideal time dilation — expressed as a set of delay durations and times across threads — is straightforward to compute if we have access to both Ops and Ops^t. Intuitively, the ideal would identify key operations (say op_d^t) in the traced execution (Ops^t) that are not in the appropriate order as per the native execution (Ops), and inject the minimum required delay into its thread ($\mathsf{Thread}(op_d^t)$) to make sure it occurs in the order prescribed by Ops. The approach trivially satisfies condition **C2** by its construction. Figure 3 demonstrates such an ideal — operations 2, 5, 6 in Thread 1, operation 4 in Thread 2, and 2^{nd}–4^{th} system operations are delayed by the minimum amount to ensure correctness.

Unfortunately, realizing such an ideal is impossible in practice since the tracing framework cannot have a priori knowledge of the native execution (Ops) or the delays caused by tracing operations ($\mathcal{T}$). While we describe our approach to achieve an approximation of the time dilation in practice next, we note that the ideal nonetheless provides a strong baseline for correctness and performance guarantees.

Practical time dilation. We now describe our practical realization of time dilation (depicted in Algorithm 1). Without loss of generality, our algorithm assumes an operation (both generated by the tracing infrastructure and native execution) is the basic unit of execution, and takes a unit time to execute — more complex operations can always be broken down into a sequence of "basic" operations. This also allows us to capture

Algorithm 1 Time Dilation: Execute operations of a traced application in a way that preserves its behavior.

Input: Ops^t, the operations in the traced execution, streamed as windows of L operations per thread.

1: **function** EXECUTETRACED(Ops^t)
2: **for** every window of L operations per thread in Ops^t **do**
3: tracingDelays $\leftarrow$ [# of tracing operations in window for each thread] ▷Array of per-thread tracing delay in W-sized window
4: maxDelay $\leftarrow \max_{\mathsf{thd} \in \mathsf{Threads}}(\mathsf{tracingDelays})$ ▷Find the most delays to a thread in this window.
5: **for** each thread thd $\in$ Threads **do**
6: $\mathsf{injectedDelay}_{\mathsf{thd}} \leftarrow (\mathsf{maxDelay} - \mathsf{tracingDelays}[\mathsf{thd}])$
7: Execute the first $L - \mathsf{injectedDelay}_{\mathsf{thd}}$ operations in thd's window, followed by a delay of $\mathsf{injectedDelay}_{\mathsf{thd}}$.
8: The last injectedDelay operations of thd are deferred to the next window. ▷To be executed first in the next window.

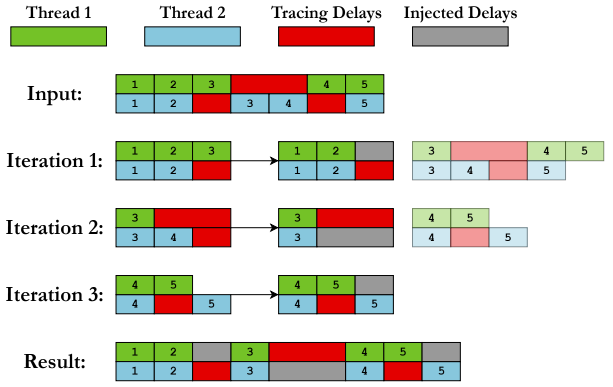

Fig. 5: Delay injection using Algorithm 1 with a window of length $L = 3$. Translucent blocks indicate operations that are yet to be processed. Using a window length greater than 1 can lead to an approximate rather than exact restoration of thread interleaving, as seen in iteration 3.

the passage of time via the execution of operations: each operation's execution advances time by a single unit.

At a high level, the algorithm considers a window of L operations at a time for every thread (Line 2) and computes the amount of delay that must be injected into the faster threads for that window based on the slowest thread — the one with the most tracing delay (Lines 3-6). It then injects the corresponding delays into the current window (Line 7, resulting in some operations for the faster threads being deferred to the next window (Line 8). The key reason this algorithm is realizable is that it only requires us to look ahead for a window of L operations at a time, rather than requiring complete knowledge of the untraced and traced executions (Ops and Ops^t) a priori.

Figure 5 shows how Algorithm 1 executes operations from two threads using a window of length $L = 3$. In each window, the algorithm identifies the thread with the largest tracing delay and injects delays into the other thread to match the largest delay. Any extra operations are pushed forward and immediately processed in the next window. The process repeats until all operations have been processed.

Practical correctness. Algorithm 1 ensures correctness under condition **C1** if the window length is $L = 1$. To see how, note that with only a single operation in each window, Algorithm 1 ensures that whenever one thread experiences tracing delays, all other threads experience the same delay at the same time. This results in a "lockstep" injection of delays, where either all threads are experiencing delays (injected or due to tracing), or all of them are executing operations. This preserves the order of key operation execution as per the native execution, satisfying condition **C2**.

Unfortunately, correctness no longer holds if we use larger windows ($L > 1$). This is evident from the execution of Iteration 3 in Figure 5, where the first and second threads' fifth operations are reordered relative to the native execution. Even so, the windowed approach guarantees that operations can only be reordered relative to native execution *within* a window — across windows, operation order is still preserved, following the same rationale as the $L = 1$ case outlined above. In other words, our windowed approach *bounds* the incorrectness (measured as the time within which operations can be reordered) to the window size.

The window size introduces an interesting tradeoff in practice — smaller window sizes require more frequent delay computations and injections, incurring higher overheads, while larger window sizes can result in possible (albeit bounded) reorders of key operations. We describe how our implementation of this algorithm in BULLETTIME navigates this tradeoff to both minimize tracing overheads and near-ideal behavior in §IV, with real-world evaluations in §V.

IV. BULLETTIME DESIGN

We now describe BULLETTIME, our realization of practical time dilation. While BULLETTIME's general approach applies to a range of simulator front-ends (e.g., Simics [33], M5 [9], etc.), we demonstrate the utility of our approach using an implementation with a Pin-based front-end [31]. This helps us test our idea on a mature software ecosystem that is widely used to build traces in computer architecture studies across simulators (e.g., zsim [43], CMP$im [26], PinPlay [38], RADISH [17], etc.) and in operating system studies (e.g., MIND [29], HotPot [44], etc.). We begin with a high-level overview of BULLETTIME's architecture (§IV-A), followed by details on how it eliminates key operations in the tracing framework (§IV-B) and how it computes and injects time dilations across application (§IV-C) and system threads (§IV-D).

A. BULLETTIME *Overview*

BULLETTIME builds on the Pin [31], where the dominant tracing overhead (i.e., contributors to $\mathcal{T}$ in our traced execution

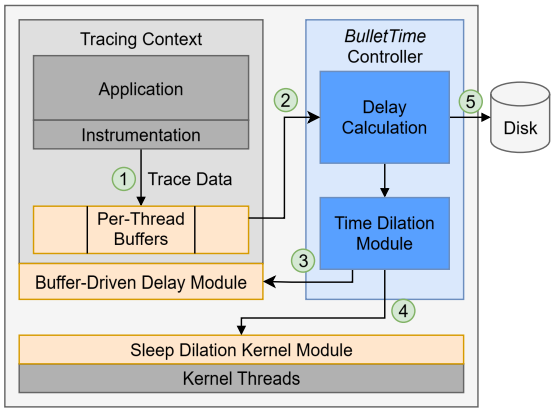

Fig. 6: BULLETTIME architecture to monitor tracing framework generated I/O operations for time dilation across system and application threads. See §IV-A for details.

model, §III-A) stems from I/O operations due to flushing traced instructions to disk. Consequently, BULLETTIME's realization of time dilation focuses on injecting delays in a manner that slows down the execution of threads that observe disproportionally lower I/O delays due to tracing.

Specifying Key State. In order to preserve application behavior, BULLETTIME must sufficiently preserve the order of key operations. This requirement suggests that users would need to themselves identify key state; i.e., the OS free memory list, application page table, lock variables, etc. Note, however, that Algorithm 1 does not require the identification of individual key operations to restore behavior, only the "key" threads that will execute key operations. BULLETTIME provides a simple interface by which users can supply a set of functions to identify key threads. When a thread executes any of these functions, BULLETTIME will include it in time dilation.

Runtime execution. Figure 6 provides an overview of BULLETTIME's overall architecture and operation. All unmodified application and system components are shown in gray. We add a new component to Pin's existing instrumentation framework — the BULLETTIME controller (shown in blue), which is responsible for delay computation and time dilation. We also add two components (shown in orange) outside the Pin framework to enforce time dilations: a Buffer-Driven Delay Module that injects delays computed by BULLETTIME controller into application threads through per-thread I/O buffers (§IV-C), and a Sleep Dilation Kernel Module that injects the delays into relevant system threads (§IV-D). At runtime, ① the tracing instrumentation (Pin) generates and stores data in internal per-thread buffers. Instead of windows of a fixed number of operations in Algorithm 1, BULLETTIME leverages the filling up of these per-thread buffers as events to both compute and inject time dilations for application threads (details in §IV-C). Specifically, when a per-thread buffers fill up, ② the delay computation module in the BULLETTIME controller calculates per-thread time dilations required across all other threads for restoring application behavior (as in Algorithm 1). These delays are then injected ③ into relevant application threads via the Buffer-Driven Delay Module when their data buffer fills up next. Since the controller intercepts the buffer fill events, it is also responsible for ④ persisting the data to disk. ⑤ The injection of delays into system threads (i.e., kernel daemons) is handled differently — the controller periodically computes a delay value for all relevant system threads, which are slowed down by extending their sleep timers via the Sleep Dilation Kernel Module (details in §IV-D).

B. Avoiding Key Operations in BULLETTIME

As we noted in §III-A, preserving application behavior under tracing requires that the tracing framework itself should not perform any modifications to key state (Condition **C1**). While it is impossible to know (and therefore avoid modifications to) key state for arbitrary studies, our BULLETTIME implementation focuses on eliminating any key operations for memory contiguity and synchronization studies, introduced in §II. We argue that the principles we use to avoid key state modifications for these classes of studies can be applied to other studies as well.

Eliminating memory allocations for contiguity studies. The vast majority of memory allocations in the underlying Pin framework stem from kernel-level caching of tracing-triggered I/O. In particular, we found that when Pin writes large amounts of traced data to disk, the kernel page cache repeatedly allocates and deallocates individual 4KB pages, severely fragmenting physical memory contiguity. To avoid this, we perform trace I/Os using the `O_DIRECT` flag to bypass the page cache and write directly to disk. Additionally, we ensure any internal buffers used by Pin to store data are backed by 2MB hugepages using `hugetlbfs` [30], and are far away from the application's allocations in terms of physical and virtual address regions. We found that such internal allocations tend to be small — a few megabytes at most — and keeping these allocations far away from application allocations eliminates any impact on application contiguity.

Eliminating synchronization over key state for synchronization studies. Our tracing framework injects minimal synchronization operations, only acquiring a brief lock to instrument code to record memory accesses. This instrumentation occurs only the first time a block of code is encountered, and is never acquired to access or modify application key state (e.g., hash-tables in our evaluated Memcached [19] application). As such, these synchronizations do not introduce any key operations.

C. Time Dilation for Application Threads

As discussed in §III-B, Algorithm 1 dilates time across faster threads to ensure they keep pace with the slowest thread; it does so by injecting delays across threads in windows of a fixed number of L operations. Realizing such an approach in BULLETTIME presents two key challenges. First, computing and realizing delays over a fixed window of operations is challenging, since instrumented operations do not tend to execute in fixed windows in practice. Second, while Algorithm 1

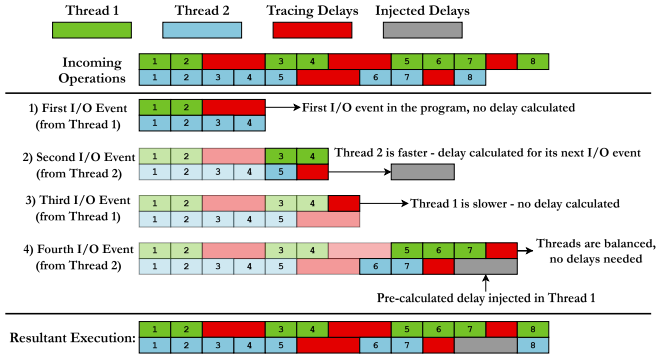

Fig. 7: BULLETTIME balances the ratio of tracing delays per unit of application progress across threads. In this example, Thread 2 executes faster than Thread 1, completing more operations before each I/O event where trace data is persisted (first and second I/O events). When BULLETTIME detects that Thread 1 is ahead of Thread 2 (at the second I/O event), it calculates a delay to inject at the *next* event to ensure it keeps pace with the slower thread (fourth I/O event). The slowest thread does not experience any additional delays under BULLETTIME (e.g., at the third I/O event).

"looks ahead" within a window of L operations to estimate tracing-induced delays per thread, it is challenging to look ahead in an execution in Pin.

To overcome these challenges, BULLETTIME makes two changes in realizing Algorithm 1. First, BULLETTIME uses the duration it takes for the per-thread tracing I/O buffer to fill up as a proxy for window size. We note, however, that since application threads may vary in the number of instrumented instructions (e.g., memory references) made per block of instructions, the filling of a per-thread buffer may correspond to different amounts of real execution time for each thread. Therefore, instead of only trying to equalize the absolute delays across threads, BULLETTIME attempts to balance *the ratio of tracing delays per unit of application "progress"*. In our theoretical model, this progress is the number of operations executed due to the untraced application. In BULLETTIME, we approximate progress as the total instructions executed by each thread, excluding Pin instrumentation overheads. Since an application thread's execution may comprise both kernel and userspace instructions, we scale their contributions by their respective instructions-per-cycle (IPC). This approach still achieves the same goal of ensuring the computed per-thread delay causes all threads to observe the same progress-to-overhead ratio as the "slowest" thread.

Second, instead of trying to "look ahead" to compute delays, BULLETTIME uses the per-thread delay calculated for the *current* window (i.e., the current buffer fill) to slow down progress in the corresponding thread's *next* window (i.e., when the buffer next fills up) via its Buffer-Driven Delay Module. To realize this, instead of performing its own I/O directly, the traced application thread delegates persistence to BULLETTIME's controller, which injects delays by simply deferring the completion notification for the corresponding disk I/O to the Buffer-Driven Delay Module. In terms of correctness, this widens the bounds on how many operations can be reordered to two windows rather than one for Algorithm 1 (§III-B). However, in practice, the time between I/O operations is relatively small ($<$ 10ms for our evaluated workloads), making this an acceptable tradeoff.

Figure 7 demonstrates an example of how BULLETTIME realizes the buffer-driven delays described above. In the example, Thread 1 experiences more tracing delays for its first time window (defined by I/O events) than Thread 2, performing its first I/O event after 2 operations instead of Thread 2's 5 operations. When Thread 2 performs its first I/O event, BULLETTIME detects this difference and computes the required delay to be injected at Thread 2's *next* I/O event, so that its rate of progress matches that of Thread 1's. Since Thread 1 continues to lag even at its second I/O event, BULLETTIME injects no extra delay for Thread 1. At Thread 2's second I/O event, BULLETTIME injects the calculated delay, restoring the rate of executions for the two threads at operation 8.

D. Time Dilation for System Threads

BULLETTIME must handle time dilation for system threads (kernel daemons) differently for two reasons. System threads are not instrumented by BULLETTIME, and therefore exhibit no tracing delays. It follows that these threads do not perform tracing-induced I/O, leaving no I/O events that can be leveraged for injecting delays.

Fortunately, kernel daemons operate at fixed periodicity — where they perform work (e.g., page compactions in `khugepaged`) every T units of time, where T is kept constant by sleeping for the time no work is performed. This provides us with a convenient way to dilate time for such threads by simply slowing down the sleep timer — this is effectively what BULLETTIME's Sleep Dilation Kernel Module does.

In more detail, BULLETTIME periodically computes the slowdown factor experienced by its slowest thread in the last period (a multiple of T in our implementation), following the approach outlined in §IV-C — this is the same factor that the system threads must be slowed down by in that period. BULLETTIME's control plane informs the Sleep Dilation Kernel Module of this slowdown factor, which in turn, extends the sleep lengths within the kernel threads to ensure its rate of progress matches that of the slowest thread.

E. Compressing Traces to Reduce I/O Overheads

While time dilation can restore application behavior, its use of delay injection can increase the already long runtime of trace collection, particularly as delays become more imbalanced (Fig. 5). To alleviate these overheads, BULLETTIME provides an option to compress traces online to reduce total I/O overheads. Intuitively, trace compression can improve runtime despite using more compute because tracing is normally bottlenecked by I/O, leading to underutilized compute resources. Thus, using more compute to compress traces can alleviate

Hardware Configuration		Data Rate	I/O BW	Ratio
CPU	DDR4 / PCIe 4.0	20-25 GB/s	4-7 GB/s	3-6×
	DDR5 / PCIe 4.0	50-70 GB/s	4-7 GB/s	7-18×
	DDR5 / PCIe 5.0	50-70 GB/s	8-14 GB/s	3-7×
GPU	GDDR6X / PCIe 4.0	1TB/s	4-7 GB/s	143-250×
	HBM2e / PCIe 5.0	2TB/s	8-14 GB/s	133-200×
	HBM3e / Infiniband	4.9TB/s	100 GB/s	49×
DDR4 / SATA		**20-25 GB/s**	**0.4-0.6 GB/s**	**33-63×**

TABLE II: Comparison of data generation rates and storage bandwidth for multiple hardware configurations. Peak data rates are calculated from the available memory bandwidth. We include memory bandwidth figures for the RTX4090, H100, and GH200 GPUs [18], [32]. Our chosen configuration (shown in bold) represents a middle ground of potential scenarios.

the I/O bottleneck. As long as compression is not overly compute-intensive, this can *improve* the compute utilization of application threads, reducing overall runtime.

In our prototype, we use zstd compression at level -7 [13], which we find provides a suitable tradeoff between compression ratio and speed. We aim to fully utilize the CPU between compression and application threads, while occasionally seeing I/O stalls. This indicates that the I/O and compute bottlenecks are balanced. We leave a more detailed exploration of compute-I/O tradeoffs for future work.

V. Evaluation

Our evaluation focuses on understanding the behavior changes caused by tracing in memory contiguity (§V-A) and synchronization (§V-B) studies, and BulletTime's effectiveness in correcting these changes.

Experimental Setup. We conduct all experiments on a machine with an Intel Core i7-8700 CPU at 3.2GHz with 6 cores and 12 threads. The storage device on our machine is a SATA-connected Samsung 870 EVO SSD. This setup represents the effect of tracing overheads on widely available commodity hardware, and constitutes a middle ground in the space of possible bandwidth ratios – as shown in Table II. We focus on the bandwidth ratio because it represents the key hardware challenge that BulletTime addresses: behavioral inaccuracies resulting from I/O stalls. BulletTime is useful in all configurations where the data generation rate significantly exceeds available I/O bandwidth.

Applications & workloads. We use three real-world applications and workloads to demonstrate the effects of tracing on contiguity behavior. (1) **Llama** [49] is an open-source large language model. We use the `llama.cpp` [20] framework to perform inference from a starting seed. As a deterministic machine learning application, one would expect Llama to have regular behavior even under traced execution. (2) **PageRank** from the GAP benchmark suite [6], a well-known graph processing algorithm. (3) **Memcached** [19] is an open-source in-memory key-value store. We use the YCSB A workload [14] **(MemA)** configured to load 10M objects 1KB in size before performing 100M requests (50/50 reads/writes). We also create a custom workload **(MemDY)** to perform 500M requests after the initial load phase, split 80/10/10 between reads, writes, and insertions. Along with object sizes uniformly distributed between 1B - 1KB, this roughly mimics Meta's ETC workload [3]. This tests BulletTime's ability to preserve contiguity for a steadily growing memory footprint.

To evaluate BulletTime's ability to preserve thread interleaving and synchronization behavior, we use a reader-writer synchronization scenario on Memcached with two clients. The first client (the reader) repeatedly makes `GET` requests to one key, while the other (the writer) repeatedly sends `UPDATE` requests to the same key — this is representative of producer-consumer patterns seen in popular cloud workloads [27], [28], [42]. Since `GET`s and `UPDATE`s execute different codepaths on the Memcached server, tracing can induce disproportionate overheads for each type of request, similar to the example in Table I. We evaluate how well time dilation in BulletTime preserves the proportion of `GET` and `UPDATE` requests completed over time.

Because key operations are so fine-grained (§IV-A), tracking BulletTime's ability to preserve the exact key operation order would induce even more interference with the behaviors we aim to preserve. Instead, we use end-to-end metrics measuring overall memory contiguity and request ratios. These metrics represent the overall effect of preserving key operation order and ultimately align better with the key behaviors themselves, for which key operations are a proxy to explain behavior.

Evaluated Baselines. We consider five baselines to understand the impact of tracing on application behavior: (1) Untraced, representing regular application execution without any tracing overhead; (2) Empty-Traced, representing a traced application execution with all memory access instructions instrumented, but without persisting any data to storage. This shows the impact of just instrumentation overhead without I/O; (3) Disk-Traced, which writes all traced data to disk using asynchronous `write()` system calls. This represents the standard approach employed in most tracing studies; (4) Disk-Traced-NC, where *NC* refers to *no-cache*. This configuration bypasses kernel I/O caching and uses hugepage-backed buffers to minimize physical memory allocations for persistent trace data (i.e., avoids key operations being generated by the tracing framework, §IV-B); (5) **BulletTime**, our tracing framework as described in §IV; (6) BT-Comp, which additionally compresses BulletTime's traces to reduce I/O requirements.

A. Preserving Memory Contiguity

We begin by examining how tracing affects memory contiguity behavior, and evaluate BulletTime's efficacy in preserving it. Figure 8 shows the physical memory contiguity of our test applications in all configurations. As in Figure 2, we show the cumulative distribution of memory covered for each page size, maximizing the amount of memory covered by larger pages. Since contiguity can vary over time, either due to memory allocations in the application or migration via kernel daemons, we examine the state of memory contiguity at the

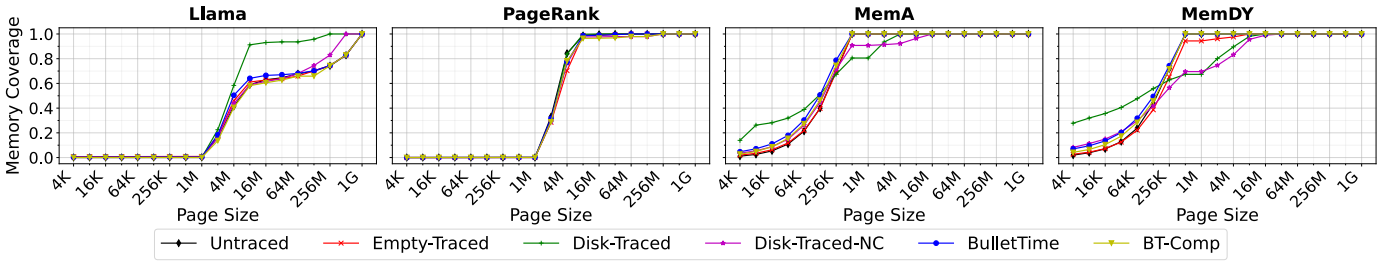

Fig. 8: Cumulative distributions of the percentage of an application's physical memory footprint that is covered by a given page size. BULLETTIME successfully restores the memory contiguity behavior of all tested applications.

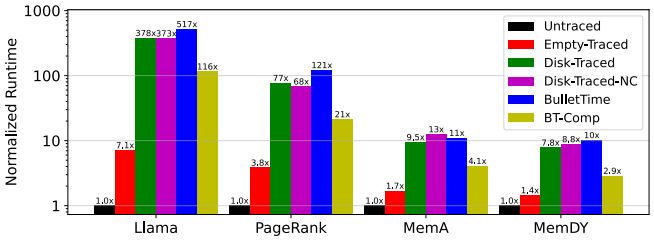

Fig. 9: Application runtime in traced configurations, normalized to the runtime of an untraced execution. BULLETTIME trades-off up to 58% runtime overhead over Disk-Traced in exchange for more accurate execution. With compression, we improve runtimes by over 2× with no accuracy loss.

	Misplaced Memory % (Total Variation Distance)				
Workload	Empty-Traced	Disk-Traced	Disk-Traced-NC	BULLETTIME	BT-Comp
Llama	6.61	32.44	20.8	9.00	6.61
PageRank	14.94	5.1	9.14	7.86	7.10
MemA	1.96	43.12	14.22	10.79	6.53
MemDY	7.41	61.29	38.42	7.91	4.75
Average	**7.73**	**35.49**	**20.65**	**8.89**	**6.25**

TABLE III: Percentage of application memory footprints placed in the wrong page size **(lower is better)**. BULLETTIME is consistently close to Untraced, while tracing configurations that write to disk are prone to significant divergence. BT-Comp further improves accuracy by reducing I/O overheads.

end of the execution, before memory is deallocated on exit. Since our workloads rarely deallocate memory before exit, the final state represents the cumulative result of all memory contiguity behavior throughout a run.

Our results show that the effects of Disk-Traced on contiguity behavior can be unpredictable. Depending on the workload, Disk-Traced can increase fragmentation, lead to improved contiguity for larger page sizes, or both. Removing kernel caching for trace data restores accuracy in small page sizes for Disk-Traced-NC. However, this approach remains inaccurate at higher page sizes due to the lack of time dilation to restore inter-thread interactions, especially with kernel threads. This is evidenced by the presence of extra 2MB pages in the MemA and MemDY workloads, indicating increased THP daemon activity. Even Empty-Traced exhibits inaccuracy here due to instrumentation delays. BULLETTIME restores this behavior by dilating kernel daemon periodicity, ensuring application contiguity closely resembles that of untraced execution.

To quantitatively evaluate contiguity preservation, we measure, in contrast to the contiguity distribution in Untraced execution, the percentage of memory covered by an incorrect page size. This can be calculated from the Total Variation Distance of the distributions in Figure 8. Table III shows our results. We see that BULLETTIME is 4× more accurate than Disk-Traced, and 2.3× more accurate than Disk-Traced-NC.

We also examine the effect of injected time dilation on the overall runtime overhead of tracing with our approach in Figure 9. BULLETTIME trades off no more than 60% extra runtime overhead compared to Disk-Traced in exchange for accurate execution, with an average runtime increase of 35%.

With trace compression, we see significant runtime improvements (>2×) with no loss in accuracy. For Llama, we find that the CPU is only 5% utilized by default. Using compression reduces I/O by 10× and improves application CPU usage to 20%, aligning with our observed runtime improvements (Figure 9). Similar utilization improvements occur for the other applications. We further see slight improvements to accuracy due to the reduced I/O overheads to balance.

B. Thread Interleaving and Synchronization

We now evaluate BULLETTIME on our synchronization study atop Memcached. The use of a dedicated reader and writer issuing requests to the same key stress tests BULLETTIME's effectiveness in preserving synchronization behavior in the midst of large overhead disparities between threads[3].

Figure 10 shows the average `GET` and `UPDATE` latencies for each configuration, sweeping over item sizes between 1-512KB. While Untraced execution shows little difference in `GET` and `UPDATE` latencies, the Empty-Traced, Disk-Traced, and Disk-Traced-NC configurations all have higher `UPDATE` latencies. This imbalance results from the difference in `GET` and `UPDATE` codepaths, with `UPDATE` requests making more memory accesses for data copies, memory allocation, etc., leading to higher tracing delays.

The effect of this asymmetry manifests in more `GET` requests being completed per unit time. Interestingly, this

[3]Each client is served by a single worker in Memcached [19].

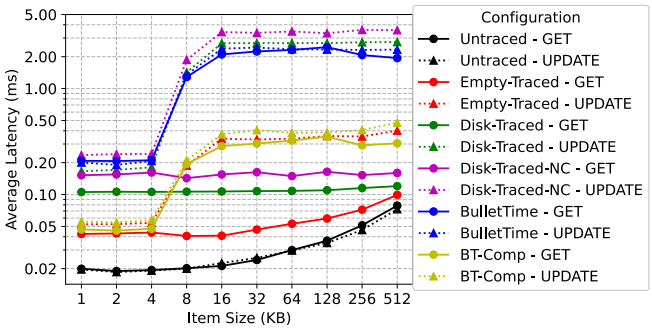

Fig. 10: Average latencies of `GET` and `UPDATE` requests in our synchronization study. BULLETTIME achieves the closest latency difference compared to untraced execution.

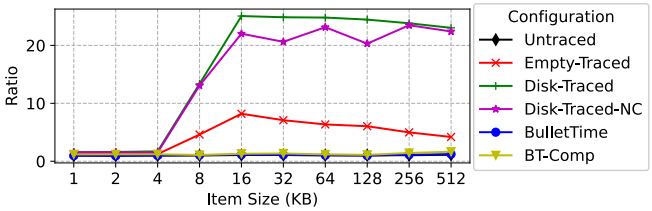

Fig. 11: Ratio of completed `GET` requests vs. completed `UPDATE` requests per unit time. Again, BULLETTIME achieves behavior closest to untraced execution.

imbalance spikes significantly as item size increases beyond 8-16KB, with Disk-Traced executing up to 25× more `GET`s than `UPDATE`s (Figure 11). This spike stems from a change in how `memcpy()` function copies incoming data to the hash table for larger items. When copying more than 8KB of data, our system's `memcpy()` switches from using AVX instructions, operating on 32-64B at a time, to `rep stosb`, which repeatedly injects microcode to store data one byte at a time. While an optimization for Untraced execution, it leads to 32-64× more memory operations that must be instrumented and recorded in the tracing framework. This change in code-path leads to an enormous imbalance in the number of `GET`s and `UPDATE`s completed per unit time for larger item sizes.

In contrast to other traced approaches, BULLETTIME successfully keeps the ratio of completed `GET`s and `UPDATE`s within 10% of native execution for all item sizes. This result is a direct consequence of BULLETTIME injecting large amounts of extra delays into the thread that serves `GET` requests (i.e., the faster thread), while injecting minimal to no delays to the thread serving `UPDATE` requests. The success of BULLETTIME's delay injection is exemplified in Figure 10, where `GET` and `UPDATE` latencies remain nearly identical. All in all, even in a workload designed to maximize the difference in tracing delays between threads, BULLETTIME still manages to accurately balance their execution speeds.

VI. Related Work

Despite almost no active study in decades, tracing-induced distortion of application execution is actually a classic problem, first identified in the 1990s for memory address tracing on single-core RISC processors running Ultrix and Mach 3.0 [1], [11]. Even on those simple platforms—with no multi-threading and limited system services—tracing overheads were problematic. The solutions of that era, such as slowing the processor's clock interrupt rate to match tracing-induced slowdown, applied primarily to single-threaded workloads. Modern systems, however, are far more complex: they depend on multi-threading, rich application–kernel interactions, and diverse system activity. These factors both amplify distortion and render prior solutions inadequate, motivating our new theoretical framework for applying time dilation more broadly—particularly now, as tracing is widely used in studies of synchronization, virtual memory, disaggregation, and beyond [17], [25], [29], [33], [40], [41], [43], [44].

More recent tracing studies ignore behavioral distortion, focusing instead on the orthogonal problem of tracing with minimal overhead. This can indirectly address distortion as, in theory, zero-overhead tracing would preserve behavior. In practice, however, reducing tracing overhead exposes another fundamental trade-off with trace *completeness*. Intel Processor Trace [24] records fine-grained control flow and timestamps, which in principle could be used to reconstruct complete instruction and memory streams, but doing so in practice would be arduous for programs with variable inputs or frequent kernel interactions. Arm CoreSight [48] provides hardware support for tracing hardware events, but since it favors dropping data over delaying execution, it is typically used only for selected subsets of operations. HMTT [23] snoops the DRAM bus to capture memory accesses with low overhead, but misses references satisfied by the cache hierarchy. All in all, these approaches sacrifice completeness for low overhead. BULLETTIME, on the other hand, avoids distortion regardless of tracing overhead or completeness. That is, its core design principles apply to the above approaches as well.

The COZ causal profiler [16] uses a scheme that works somewhat inversely to BULLETTIME. COZ calculates the potential application speedup from optimizing one section of code by proportionally slowing all other sections of the application. BULLETTIME handles the effects of slowing down one thread of a program by equally slowing all other threads.

VII. Conclusion

Tracing frameworks inject asymmetric delays across traced application threads when persisting traced data, which can change application behavior relative to untraced executions. We formally define notions of correctness for traced executions, and propose a novel *time dilation* approach to achieve this correctness. We have realized time dilation in BULLETTIME, a tracing framework built atop Pin. Our evaluations of memory contiguity and synchronization studies over real-world workloads show that existing tracing approaches can cause application behavior to deviate by as much as 20× relative to untraced execution. In contrast, BULLETTIME deviates by at most 10% for all evaluated workloads.

REFERENCES

[1] M. Accetta, R. Baron, W. Bolosky, D. Golub, R. Rashid, A. Tevanian, and M. Young, "Mach: A new kernel foundation for unix development," 1986.

[2] A. Arcangeli, "Transparent hugepage support," in *KVM forum*, vol. 9, 2010.

[3] B. Atikoglu, Y. Xu, E. Frachtenberg, S. Jiang, and M. Paleczny, "Workload analysis of a large-scale key-value store," in *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE joint international conference on Measurement and Modeling of Computer Systems*, 2012, pp. 53–64.

[4] K. Banker, D. Garrett, P. Bakkum, and S. Verch, *MongoDB in action: covers MongoDB version 3.0*. Simon and Schuster, 2016.

[5] T. W. Barr, A. L. Cox, and S. Rixner, "Translation caching: skip, don't walk (the page table)," *ACM SIGARCH Computer Architecture News*, vol. 38, no. 3, pp. 48–59, 2010.

[6] S. Beamer, K. Asanović, and D. Patterson, "The gap benchmark suite," *arXiv preprint arXiv:1508.03619*, 2015.

[7] R. Bera, K. Kanellopoulos, A. Nori, T. Shahroodi, S. Subramoney, and O. Mutlu, "Pythia: A customizable hardware prefetching framework using online reinforcement learning," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 1121–1137.

[8] A. Bhattacharjee, "Translation-triggered prefetching," in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, 2017, pp. 63–76.

[9] N. L. Binkert, R. G. Dreslinski, L. R. Hsu, K. T. Lim, A. G. Saidi, and S. K. Reinhardt, "The m5 simulator: Modeling networked systems," *Ieee micro*, vol. 26, no. 4, pp. 52–60, 2006.

[10] J. Carlson, *Redis in action*. Simon and Schuster, 2013.

[11] J. B. Chen, D. W. Wall, and A. Borg, "Software methods for system address tracing: implementation and validation," *WRL Research Report 94/6*, 1994.

[12] D. W. Clark and J. S. Emer, "Performance of the vax-11/780 translation buffer: Simulation and measurement," *ACM Transactions on Computer Systems (TOCS)*, vol. 3, no. 1, pp. 31–62, 1985.

[13] Y. Collet and M. Kucherawy, "Zstandard compression and the application/zstd media type," Tech. Rep., 2018.

[14] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the 1st ACM symposium on Cloud computing*, 2010, pp. 143–154.

[15] G. Cox and A. Bhattacharjee, "Efficient address translation for architectures with multiple page sizes," *ACM SIGPLAN Notices*, vol. 52, no. 4, pp. 435–448, 2017.

[16] C. Curtsinger and E. D. Berger, "Coz: Finding code that counts with causal profiling," in *Proceedings of the 25th Symposium on Operating Systems Principles*, 2015, pp. 184–197.

[17] J. Devietti, B. P. Wood, K. Strauss, L. Ceze, D. Grossman, and S. Qadeer, "Radish: always-on sound and complete ra d etection in s oftware and h ardware," *ACM SIGARCH Computer Architecture News*, vol. 40, no. 3, pp. 201–212, 2012.

[18] J. Evans, M. Andersch, V. Sethi, G. Brito, and V. Mehta, "Nvidia grace hopper superchip architecture in-depth," Jul 2025. [Online]. Available: https://developer.nvidia.com/blog/nvidia-grace-hopper-superchip-architecture-in-depth

[19] B. Fitzpatrick, "Distributed caching with memcached," *Linux journal*, vol. 2004, no. 124, p. 5, 2004.

[20] G. Gerganov, "llama.cpp," https://github.com/ggml-org/llama.cpp, 2025.

[21] S. R. Goldschmidt and J. L. Hennessy, "The accuracy of trace-driven simulations of multiprocessors," *ACM SIGMETRICS Performance Evaluation Review*, vol. 21, no. 1, pp. 146–157, 1993.

[22] F. Guvenilir and Y. N. Patt, "Tailored page sizes," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 900–912.

[23] Y. Huang, L. Chen, Z. Cui, Y. Ruan, Y. Bao, M. Chen, and N. Sun, "Hmtt: A hybrid hardware/software tracing system for bridging the dram access trace's semantic gap," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 11, no. 1, pp. 1–25, 2014.

[24] Intel, "Intel® processor trace," https://edc.intel.com/content/www/us/en/design/products/platforms/processor-and-core-i3-n-series-datasheet-volume-1-of-2/001/intel-processor-trace/, 2015.

[25] R. Jagtap, S. Diestelhorst, and A. Hansson, "Elastic traces for fast and accurate system performance exploration," in *2016 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2016, pp. 147–148.

[26] A. Jaleel, R. S. Cohn, C.-K. Luk, and B. Jacob, "Cmp$im: A pin-based on-the-fly multi-core cache simulator," in *Proceedings of the Fourth Annual Workshop on Modeling, Benchmarking and Simulation (MoBS), co-located with ISCA*, 2008, pp. 28–36.

[27] A. Khandelwal, Y. Tang, R. Agarwal, A. Akella, and I. Stoica, "Jiffy: elastic far-memory for stateful serverless analytics," in *Proceedings of the Seventeenth European Conference on Computer Systems*, ser. EuroSys '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 697–713. [Online]. Available: https://doi.org/10.1145/3492321.3527539

[28] A. Klimovic, Y. Wang, P. Stuedi, A. Trivedi, J. Pfefferle, and C. Kozyrakis, "Pocket: Elastic ephemeral storage for serverless analytics," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, Oct. 2018, pp. 427–444. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/klimovic

[29] S.-s. Lee, Y. Yu, Y. Tang, A. Khandelwal, L. Zhong, and A. Bhattacharjee, "Mind: In-network memory management for disaggregated data centers," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, 2021, pp. 488–504.

[30] H. Lu, K. Doshi, R. Seth, and J. Tran, "Using hugetlbfs for mapping application text regions," in *Proceedings of the Linux Symposium*, vol. 2, 2006, pp. 75–82.

[31] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood, "Pin: building customized program analysis tools with dynamic instrumentation," *Acm sigplan notices*, vol. 40, no. 6, pp. 190–200, 2005.

[32] W. Luo, R. Fan, Z. Li, D. Du, Q. Wang, and X. Chu, "Benchmarking and dissecting the nvidia hopper gpu architecture," in *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2024, pp. 656–667.

[33] P. S. Magnusson, M. Christensson, J. Eskilson, D. Forsgren, G. Hallberg, J. Hogberg, F. Larsson, A. Moestedt, and B. Werner, "Simics: A full system simulation platform," *Computer*, vol. 35, no. 2, pp. 50–58, 2002.

[34] M. Martonosi, A. Gupta, and T. Anderson, "Memspy: Analyzing memory system bottlenecks in programs," *ACM SIGMETRICS Performance Evaluation Review*, vol. 20, no. 1, pp. 1–12, 1992.

[35] ——, "Effectiveness of trace sampling for performance debugging tools," *ACM SIGMETRICS Performance Evaluation Review*, vol. 21, no. 1, pp. 248–259, 1993.

[36] S. Mirbagher-Ajorpaz, E. Garza, G. Pokam, and D. A. Jiménez, "Chirp: Control-flow history reuse prediction," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 131–145.

[37] N.-C. Papadopoulos, S. Psomadakis, V. Karakostas, N. Koziris, and D. N. Pnevmatikatos, "Design, implementation and evaluation of the sv-napot extension on a risc-v processor," *arXiv preprint arXiv:2406.17802*, 2024.

[38] H. Patil, C. Pereira, M. Stallcup, G. Lueck, and J. Cownie, "Pinplay: a framework for deterministic replay and reproducible analysis of parallel programs," in *Proceedings of the 8th annual IEEE/ACM international symposium on Code generation and optimization*, 2010, pp. 2–11.

[39] B. Pham, A. Bhattacharjee, Y. Eckert, and G. H. Loh, "Increasing tlb reach by exploiting clustering in page translations," in *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2014, pp. 558–567.

[40] B. Pham, V. Vaidyanathan, A. Jaleel, and A. Bhattacharjee, "Colt: Coalesced large-reach tlbs," in *2012 45th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2012, pp. 258–269.

[41] B. Pham, J. Veselỳ, G. H. Loh, and A. Bhattacharjee, "Large pages and lightweight memory management in virtualized environments: Can you have it both ways?" in *Proceedings of the 48th International Symposium on Microarchitecture*, 2015, pp. 1–12.

[42] Q. Pu, S. Venkataraman, and I. Stoica, "Shuffling, fast and slow: Scalable analytics on serverless infrastructure," in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. Boston, MA: USENIX Association, Feb. 2019, pp. 193–206. [Online]. Available: https://www.usenix.org/conference/nsdi19/presentation/pu

[43] D. Sanchez and C. Kozyrakis, "Zsim: Fast and accurate microarchitectural simulation of thousand-core systems," *ACM SIGARCH Computer architecture news*, vol. 41, no. 3, pp. 475–486, 2013.

[44] Y. Shan, S.-Y. Tsai, and Y. Zhang, "Distributed shared persistent memory," in *Proceedings of the 2017 Symposium on Cloud Computing*, 2017, pp. 323–337.

[45] Z. Shi, A. Jain, K. Swersky, M. Hashemi, P. Ranganathan, and C. Lin, "A hierarchical neural model of data prefetching," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 861–873.

[46] R. T. Short and H. M. Levy, "A simulation study of two-level caches," *ACM SIGARCH Computer Architecture News*, vol. 16, no. 2, pp. 81–88, 1988.

[47] K. Sriram, I. Karageorgos, X. Wen, J. Veselỳ, N. Lindsay, M. Wu, L. Khazan, R. P. Pothukuchi, R. Manohar, and A. Bhattacharjee, "Halo: A hardware–software co-designed processor for brain–computer interfaces," *Ieee micro*, vol. 43, no. 3, pp. 64–72, 2023.

[48] A. P. Su, J. Kuo, K.-J. Lee, J. Huang, G.-A. Jian, C.-A. Chien, J.-I. Guo, and C.-H. Chen, "Multi-core software/hardware co-debug platform with arm coresight™, on-chip test architecture and axi/ahb bus monitor," in *Proceedings of 2011 International Symposium on VLSI Design, Automation and Test*. IEEE, 2011, pp. 1–6.

[49] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[50] R. Uhlig, D. Nagle, T. Mudge, S. Sechrest, and J. Emer, "Instruction fetching: Coping with code bloat," *ACM SIGARCH Computer Architecture News*, vol. 23, no. 2, pp. 345–356, 1995.

[51] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. Steely Jr, and J. Emer, "Ship: Signature-based hit predictor for high performance caching," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, 2011, pp. 430–441.

[52] J. Xu, M. Dong, Q. Tian, Z. Tian, T. Xin, and H. Chen, "Asyncfs: Metadata updates made asynchronous for distributed filesystems with in-network coordination," *arXiv preprint arXiv:2410.08618*, 2024.

[53] Z. Yan, D. Lustig, D. Nellans, and A. Bhattacharjee, "Translation ranger: Operating system support for contiguity-aware tlbs," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 698–710.

[54] R. Zhang, S. Biswas, V. Balaji, M. D. Bond, and B. Lucia, "Peacenik: Architecture support for not failing under fail-stop memory consistency," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 317–333.

[55] K. Zhao, K. Xue, Z. Wang, D. Schatzberg, L. Yang, A. Manousis, J. Weiner, R. Van Riel, B. Sharma, C. Tang *et al.*, "Contiguitas: The pursuit of physical memory contiguity in datacenters," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.